

그가 다모양문법무말배고덕배문결문도문배문결문드데t 리보라대덕o 문말및

Hbz개t e 개개t B Q 것갈 hz~z~갈 감갈 | 공경kz 감z 것 S 감개만 감공B Q 것e 개공개 과갈 감 | V5 감개 n 감z 경Vz 경개 열 e 갈감 개
b가 감h 개감t 3m 감개 개개개 k 가과갈 계
Hn 것갈 개갈 개개 b z 개개개 B Q 각개
6 Hh 개개 5QSe e z 것개 5Tz 고 / l aVè g T Oo 개한 3f 개>3>8>A
H >D
H/SO 6

Yi aY구 갈i 공경개가도 5
5
H
U교공과형 / 0 0
/ 0
H
: 5 H
>5
A5 H H 과개
H H /U교공과형 /Yi a 5U교공과형
Yi a 3 5

T:H

3 리보라대덕o 문말4

Qa H
H
/U것개개 3 3 5 6
/k가개개 5 / H(Q R (0
/Q고갈공개 3 3

예: 전자상거래 지식 그래프



/ 2 0 H

사용 사례 1: 추천 시스템
"사용자 10이 본 상품과 유사한 상품을 찾되,

같은 카테고리에 속하고 가격이 50,000~150,000 범위인 것"

사용 사례 2: 중복 검출

"이 상품과 설명이 비슷한 다른 상품들 (가능한 중복 상품)"

사용 사례 3: 이상 탐지

"이 시리즈 제품들과 외형이 비슷한 상품들 (위조 제품 탐지)"

급 H

시간 T에 처리할 쿼리:

└ (사용자1의 선호 벡터, 카테고리=전자제품, 가격=50K~150K)

└ (사용자2의 선호 벡터, 카테고리=의류, 가격=30K~80K)

└ (사용자3의 선호 벡터, 카테고리=가구, 가격=100K~500K)

└ ... (수천 개 더)

└ (사용자N의 선호 벡터, ...)

H

3

H : 3A ((벡 A
/i hl 0백 벡

T:l [p z r 3분 문서z 법 문서라 더 분

Yi a 5

[1

단계 1: 워크로드 분석

최근 1주일 쿼리 로그 분석
- 어떤 카테고리가 자주 검색되나?
- 어떤 가격 대역이 함께 나타나?
- 어떤 벡터 영역이 많이 사용?

↓

단계 2: 파티셔닝 결정

파티션 설계
파티션 1: 전자제품 + 50K~150K
파티션 2: 의류 + 30K~80K
파티션 3: 가구 + 100K~500K
...

↓

단계 3: 데이터 재조직

상품들을 파티션에 따라 재배치
파티션 1 벡터들을 메모리/SSD에
함께 로드하여 접근성 향상

기존 시스템: 카테고리별로만 저장

Q1: (벡터v1, 카테고리="전자제품", 가격=50K~150K)

└ 전자제품 파티션 스캔

Q2: (벡터v2, 카테고리="전자제품", 가격=200K~300K)

└ 전자제품 파티션 스캔 (다시)

Q3: (벡터v3, 카테고리="의류", 가격=30K~80K)

→ 같은 파티션을 여러 번 스캔 → I/O 비효율

Q1: (벡터v1, 카테고리="전자제품", 가격=50K~150K)
└ (전자제품, 50K~150K) 파티션 스캔

Q2: (벡터v2, 카테고리="전자제품", 가격=200K~300K)
 ↳ (전자제품, 200K~300K) 파티션 스캔

Q3: (벡터v3, 카테고리="의류", 가격=30K~80K)
└ (의류, 30K~80K) 파티션 스캔

→ 각 쿼리가 필요한 파티션만 정확히 선택 → I/O 최소화

[

3

H

```
Q1: SELECT * FROM products
    WHERE embedding <-> q1_vec < 1.0 AND price < 100K
```

```
Q2: SELECT * FROM products
    WHERE embedding <-> q2_vec < 1.0 AND price < 100K
```

- 같은 상품들이 두 쿼리의 상위 후보에 나타날 가능성 높음
- 이미 q1에서 계산한 거리를 q2에 재사용 가능 → 계산 비용 절반으로 감소

[

Q1 [0.3, -0.1, 0.5] $\neg$
 Q2 [0.3, 0.0, 0.5] $\vdash$ 클러스터 A (가까움)
 Q3 [0.4, -0.1, 0.6] $\neg$

Q4 [0.8, 0.9, -0.2] $\neg$
Q5 [0.7, 0.8, 0.0] $\vdash$ 클러스터 B (가까움)
Q6 [0.9, 1.0, -0.1] $\neg$

- 같은 벡터 영역 검색
- 후보 세트가 겹침 → 거리 계산 재사용

- 다른 벡터 영역 검색
- 후보 세트 다름 → 별도 계산

H

Q1 거리 계산: 1000개 벡터 × 1024차원 = 1,024,000 연산
Q2 거리 계산: 1000개 벡터 × 1024차원 = 1,024,000 연산
Q3 거리 계산: 1000개 벡터 × 1024차원 = 1,024,000 연산
합계: 3,072,000 연산

Q1, Q2, Q3 (클러스터 A에 속함)

- 한 번에 처리: 1000개 벡터 × 1024차원 = 1,024,000 연산
- 결과를 Q1, Q2, Q3 모두에 재사용

합계: 1,024,000 연산

절감: 66% (3배 빠름)

T:M 3g 더는 말바라하리다. 본결의 문배방 4

[

쿼리: 벡터 유사도 + 속성 필터 조건

```
예: WHERE embedding <-> query_vec < 1.0 AND price < 100K
```

속성 필터의 선택도에 따라 최적 처리 순서가 달라진다:

1. 벡터 우선 (Post-Filtering):

벡터 인덱스 → 상위 1000개 찾기 → 가격 필터 적용

장점: 벡터 검색 자체가 빠름

단점: 1000개 벡터 중 대부분이 가격 필터에 탈락하면 낭비

적합 상황: 속성 필터의 선택도가 높을 때 (80% 이상)

2. 속성 우선 (Pre-Filtering):

가격 인덱스 → 100K 이하의 상품들 찾기 → 벡터 유사도 계산

장점: 미리 필터된 작은 세트에만 벡터 계산 수행

단점: 속성 필터로 걸러진 상품이 적으면 소수 세트만 남음

적합 상황: 속성 필터의 선택도가 낮을 때 (10% 이하)

p z r

속성 필터의 선택도 학습:

과거 쿼리 분석:

└ (카테고리=전자제품, 가격<100K) → 선택도 5% (속성 우선!)

└ (카테고리=의류, 가격<100K) → 선택도 80% (벡터 우선!)

└ (카테고리=가구, 가격<100K) → 선택도 25% (하이브리드!)

L . . .

런타임 최적화:

새 쿼리 (카테고리=전자제품, ...) 들어옴

→ "전자제품 + 가격 필터"의 선택도 = 5%

→ 속성 우선 인덱스 선택

→ 최대한 빨리 필터링된 세트를 구성

T:0

H

HQ격점가 / 3 3 0

H /: 8e 0

HC: >

H

H: 88_H 83888 6
$$H \quad \quad \quad 2$$

H

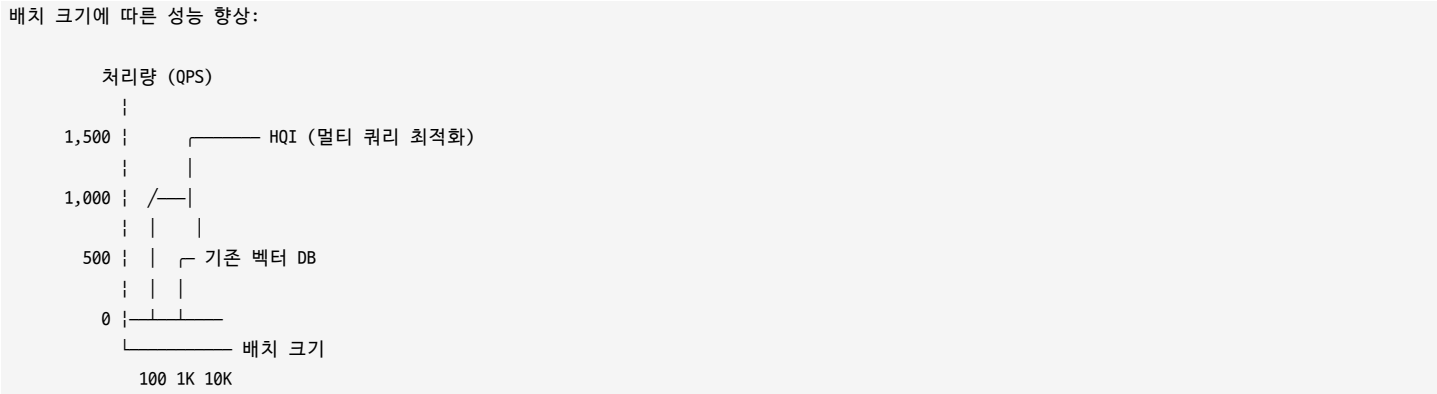
/것게거결견겸갑교계것

TR/0

 $\gamma_i \alpha /$

		TR	Yi a	
~~~~~				
: 88	C8 i hl	: C8 i hl	: C8 i hl	A:
: 888	BC i hl	: >8 i hl	: 88 i hl	>D
: 8888	B8 i hl	: 88 i hl	F88 i hl	>8

H



H

: 5  
>5  
A5 / 0

백 e i g

백 8g

T:Q

	결과 개명	Yi a
~~~~~		
	/ 0	/ 0
	3	

Yi a ((결과 개명 H

현재 HQI:

- 과거 쿼리 결과에서 선택도 학습 (결과 기반)
- 새 쿼리에 "유사한" 과거 쿼리의 선택도 재사용
- 문제: 과거 데이터가 없으면 부정확

Exqutor 적용:

- 정확한 카디널리티 추정으로 선택도 계산 (추정 기반)
- 새로운 속성 필터 조건도 정확히 예측 가능
- 장점: 과거 데이터 불필요, 항상 정확

통합 시스템:

- 배치 들어옴
- ↓
- 쿼리별 실행 계획 최적화 (Exqutor ECQ0)
- └─ 정확한 카디널리티로 최적의 조인 순서 결정
 - └─ 각 쿼리의 실행 비용 추정
- ↓
- 배치 수준 최적화 (HQI)
- └─ 쿼리 클러스터링 (유사한 쿼리 그룹화)
 - └─ 계산 공유 (벡터 거리 계산 재사용)
 - └─ 적응적 인덱스 선택 (선택도 기반)

T:S

: 5

5

H

(

:

5

/

0

3

5

>5Q

각각

H

Q

각각

5

3

5

각각

각각

각각

각각

0

5

A5

H

(

/

0

(

5

/

H

Qf T

Qf T

NB50

3

5

B5

HYi a

(

(

3

5

4

5

C5

HC: >

5

/: 88820

3e i g

5